

Docker

Name: Noora Najeeb Al-tamimi

▪ What is Docker?

Docker is an open platform for developing, shipping, and running applications. Docker enable you to:

- Separate your applications from your infrastructure so you can deliver software quickly.
- You can manage your infrastructure in the same ways you manage your applications.
- Take advantage of Docker's methodologies for shipping, testing, and deploying code, you can significantly reduce the delay between writing code and running it in production.

▪ The Docker platform:

Docker provides the ability to package and run application in a loosely isolated environment called a container. The isolation and security lets you run many containers simultaneously on a given host. Containers are lightweight and contain everything needed to run the application, so you don't need to rely on what's installed on the host.

You can share containers while you work, and be sure that everyone you share with gets the same container that works in the same way.



Docker provides tooling and a platform to manage the lifecycle of your containers:

- Develop your application and its supporting components using containers.
- The container becomes the unit for distributing and testing your application .
- When you're ready, develop your application into your production environment, as a container or an orchestrated service. This works the same whether your production

environment is a local data center, a cloud provider, or a hybrid of the two

- **Docker Architecture:**

Docker uses a client-server architecture. The Docker daemon which does the heavy lifting of building, running, and distributing your Docker containers. The Docker client and daemon can run on the same system, or you can connect a Docker client to a remote Docker daemon. The Docker client and daemon communicate using a REST API, over UNIX sockets or a network interface. Another Docker client is Docker Compose, that lets you work with applications consisting of a set of containers.

- **The Docker daemon:**

The Docker daemon (dockerd) listens for Docker API requests and manages Docker objects such as image, containers, networks, and volumes. A daemon can also communicate with other daemons to manage Docker services.

- **Docker client:**

The Docker client (docker) is the primary way many Docker users interact with Docker. When you use commands such as (docker run) , the client sends these commands to dockerd, which carries them out. The docker command uses the Docker API. The Docker client can communicate with more than one daemon.

- **Docker desktop:**

Docker Desktop is an easy-to-install application for your Mac, Windows, or Linux environment that enables you to build and share containerized applications and microservices. Docker Desktop includes the Docker daemon(dockerd), the Docker client(docker), Docker Compose, Docker Content Trust, Kubernetes, and Credential Helper.

- **Docker registries:**

A Docker registry stores Docker images. Docker Hub is a public registry that anyone can use, and Docker looks for images on Docker Hub by default. You can even run your own private registry.

- When you use the `docker pull` or `docker run` command, Docker pulls the required images from your configured registry. When you use the `docker push` command, Docker pushes your image to your configured registry.
- **Docker object:**
When you use Docker, you are creating and using images, containers, networks, volumes, plugins, and other objects.
- **Images:**
An image is read-only template with instructions for creating a Docker container. Often, an image is based on another image, with some additional customization. For example, you may build an image that is based on the Ubuntu image but includes the Apache web server and your application, as well as the configuration details needed to make your application run.
- **Containers:**
A container is a runnable instance of an image. You can create, start, stop, move, or delete a container using the Docker API or CLI. You can connect a container to one or more networks, attach storage to it or even create a new image based on its current.

➤ **Docker Architecture:**



